ЗВІТ з практичної роботи № 13

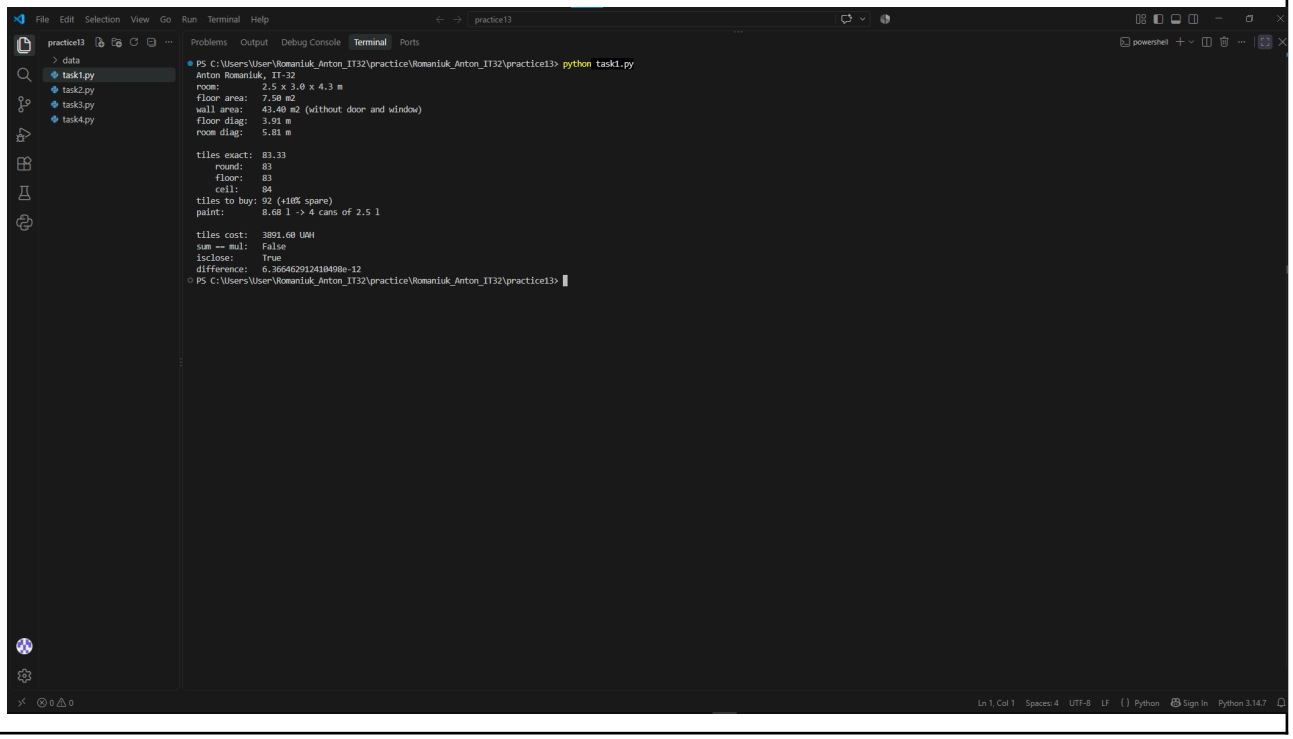
Студент: Anton Romaniuk

Група: IT-32

Завдання 1. Калькулятор ремонту

Програма task1.py за константами (розміри кімнати, дверей, вікна, плитки та її ціна) обчислює площі підлоги й стін, діагоналі через math.hypot і math.dist, кількість плиток трьома способами округлення (round, math.floor, math.ceil), кількість фарби та вартість двома способами (множення і додавання в циклі) з порівнянням через == і math.isclose.

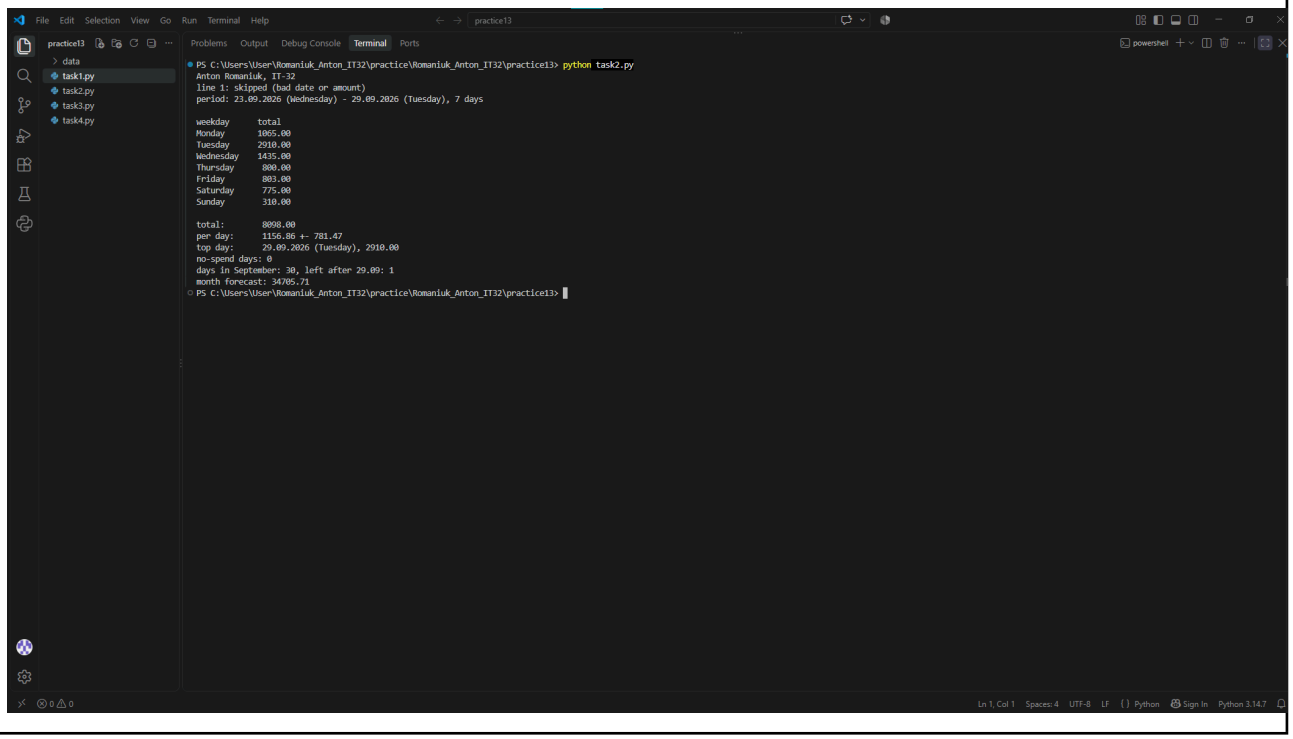
python task1.py



Завдання 2. Коли ви витрачаєте гроші

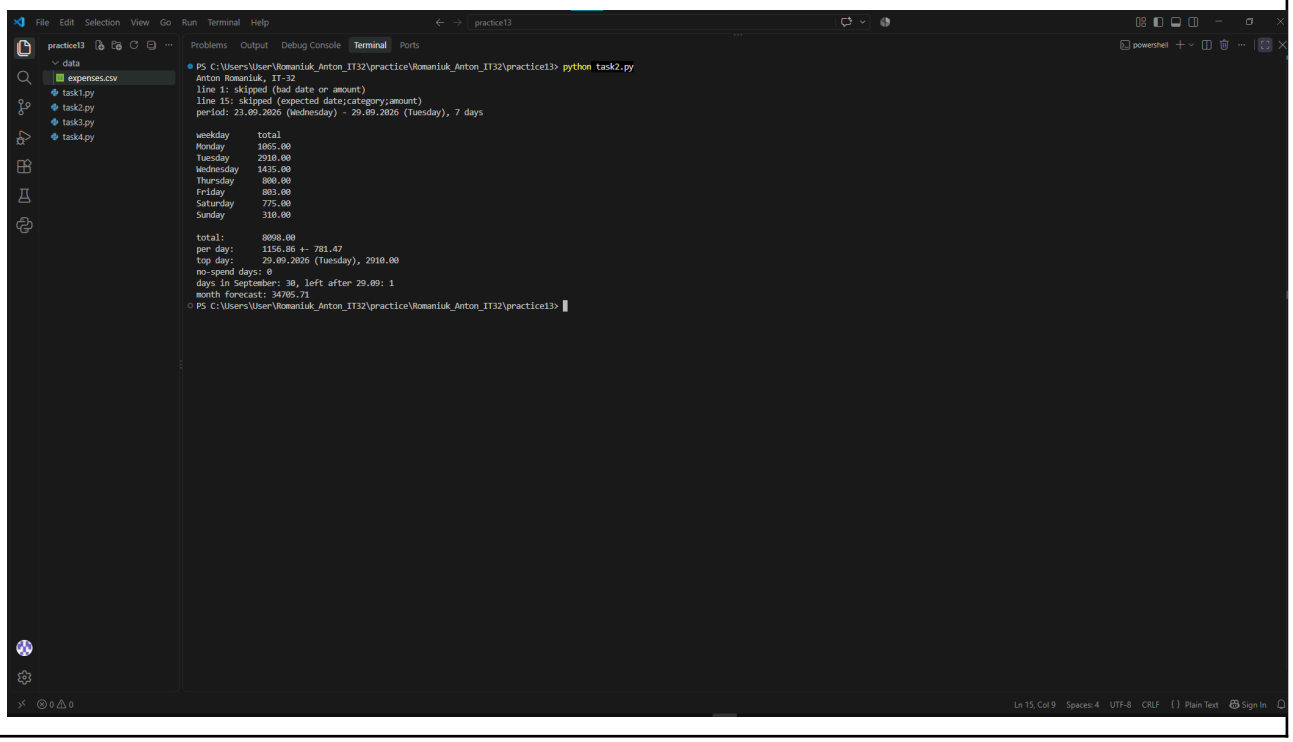
Програма task2.py читає data/expenses.csv і аналізує витрати **за датами**: період і кількість днів, суми за днями тижня, загальну суму (math.fsum), середнє й стандартне відхилення за день (math.sqrt), день з найбільшими витратами, дні без витрат, кількість днів у місяці останньої дати та прогноз витрат на місяць. Коментарі та порожні рядки пропускаються, зіпсовані — пропускаються з попередженням у sys.stderr. Кількість днів у місяці рахується через різницю дат, а не через таблицю.

```
python task2.py
```



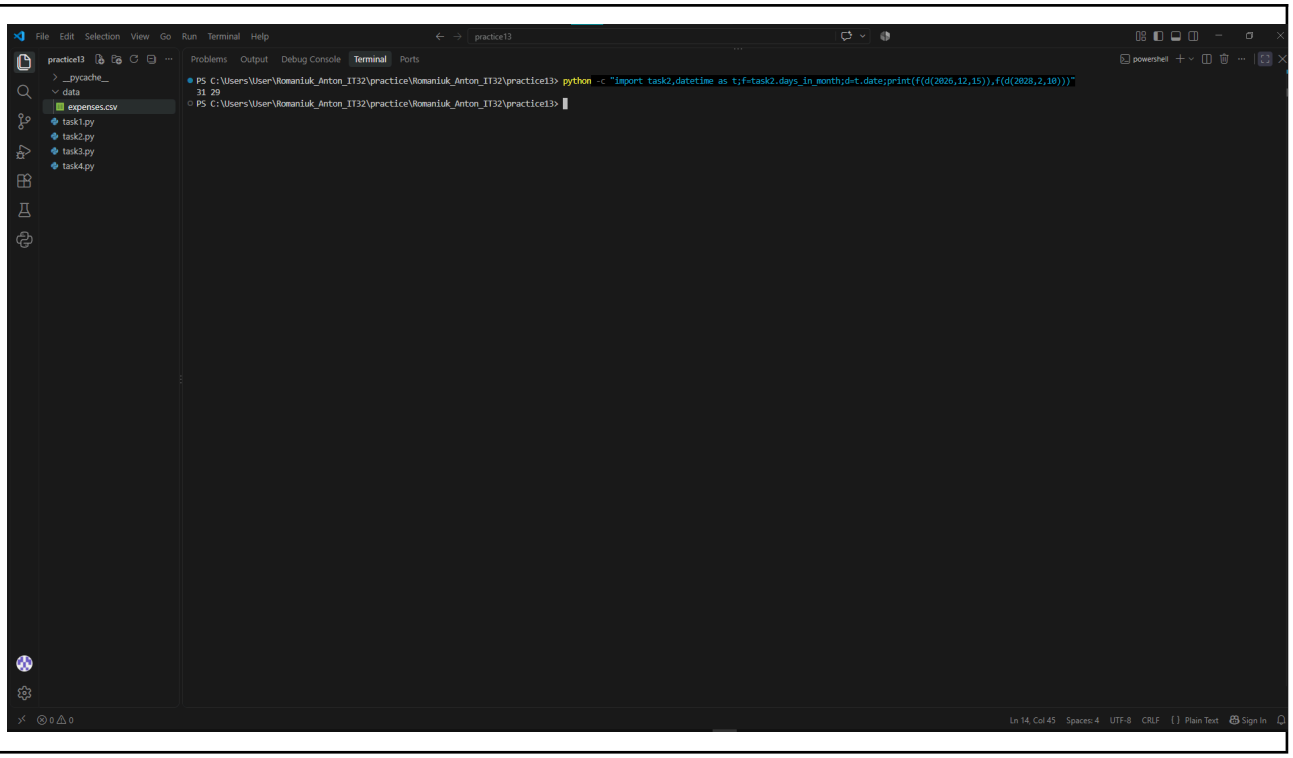
Попередження про зіпсовані рядки:

```
python task2.py
```



Перевірка кількості днів у місяці для 2026-12-15 (перехід року) та 2028-02-10 (високосний рік):

```
python -c "import task2,datetime as t;f=task2.days_in_month;d=t.date;print(f(d(2026,12,15)),f(d(2028,2,10)))"
```

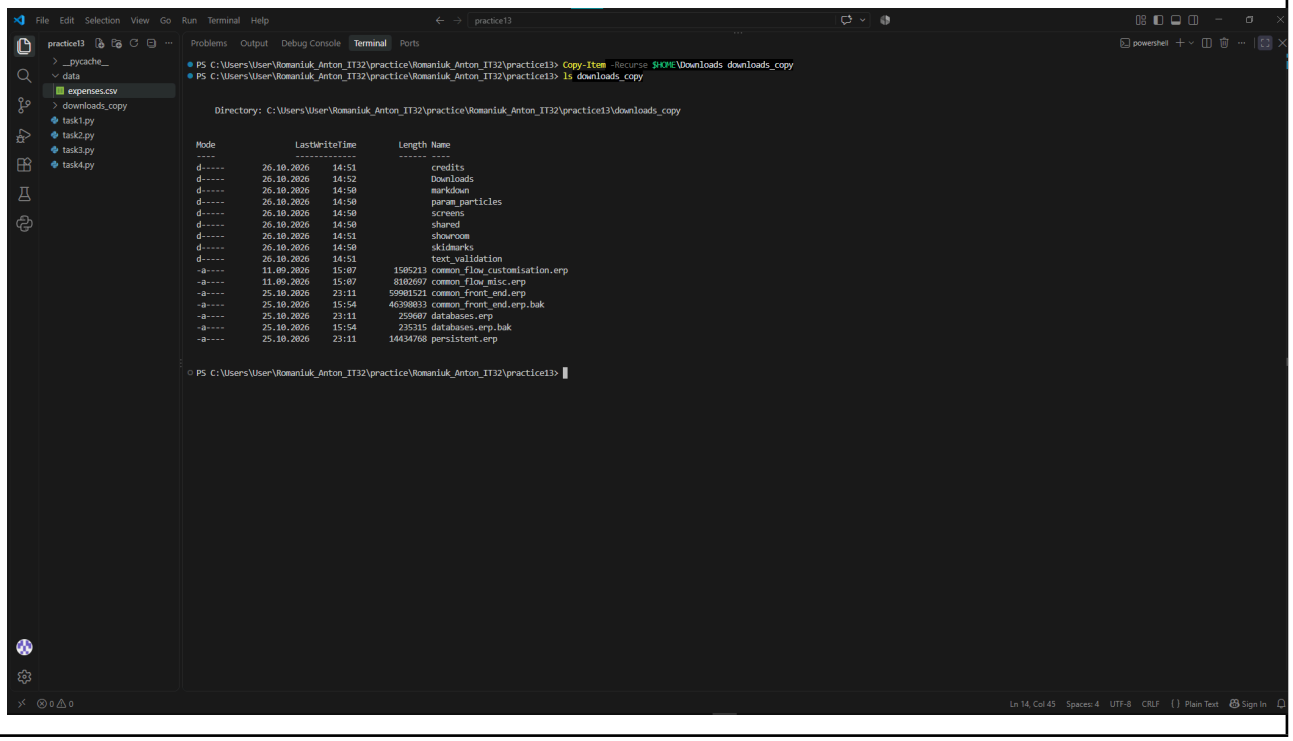


Завдання 3. Сортувальник файлів

Програма task3.py [--dry-run] <dir> розкладає файли каталогу по підкаталогах за місяцем останньої зміни (2026-09 тощо). Дата береться з Path.stat().st_mtime через datetime.fromtimestamp; розмір виводиться в зручному вигляді (одиниця визначається через math.log(size, 1024)); файли йдуть від найстарішого до найновішого. Перед переміщенням перевіряються **всі** конфлікти, після — записується звіт у reports/organize_YYYYMMDD_HHMM.txt поруч зі скриптом.

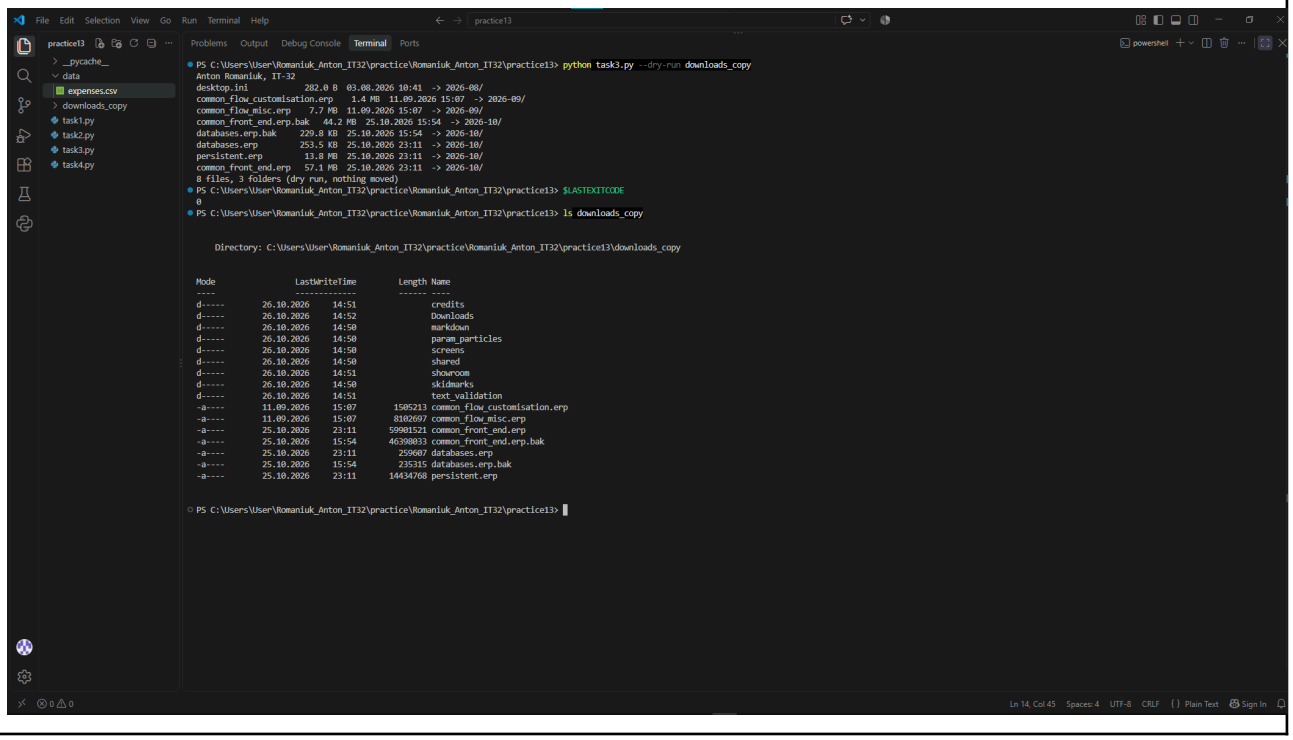
Підготовка копії каталогу:

```
Copy-Item -Recurse $HOME\Downloads downloads_copy
ls downloads_copy
```



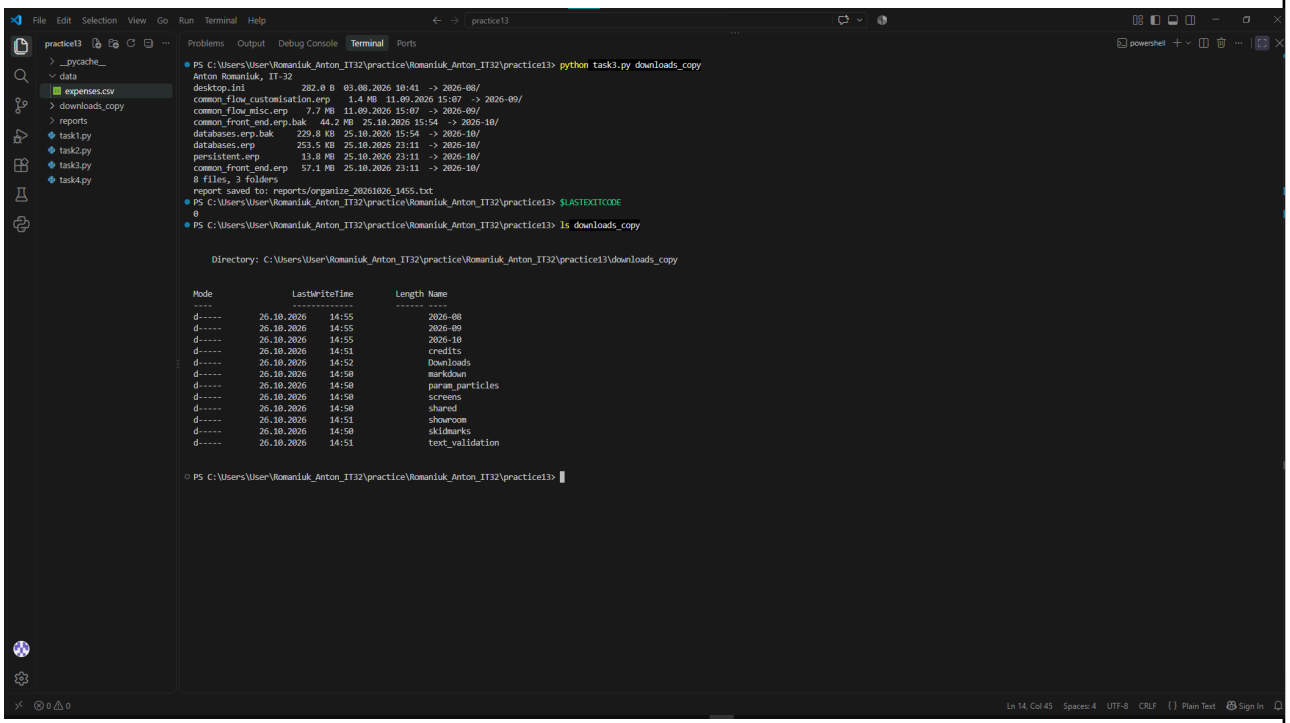
Прогін 1. Режим --dry-run (нічого не переміщується, код 0):

```
python task3.py --dry-run downloads_copy
$LASTEXITCODE
ls downloads_copy
```

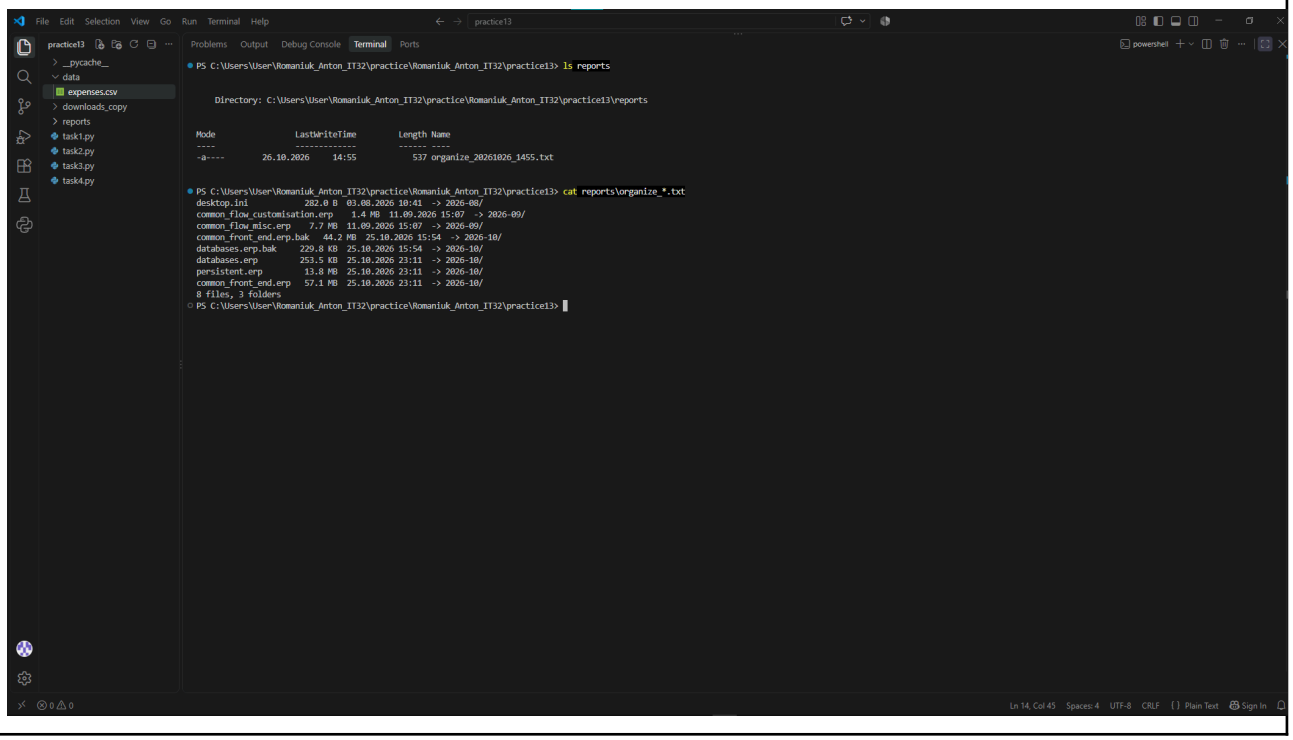


Прогін 2. Реальний запуск (файли переміщено, збережено звіт, код 0):

```
python task3.py downloads_copy
$LASTEXITCODE
ls downloads_copy
```

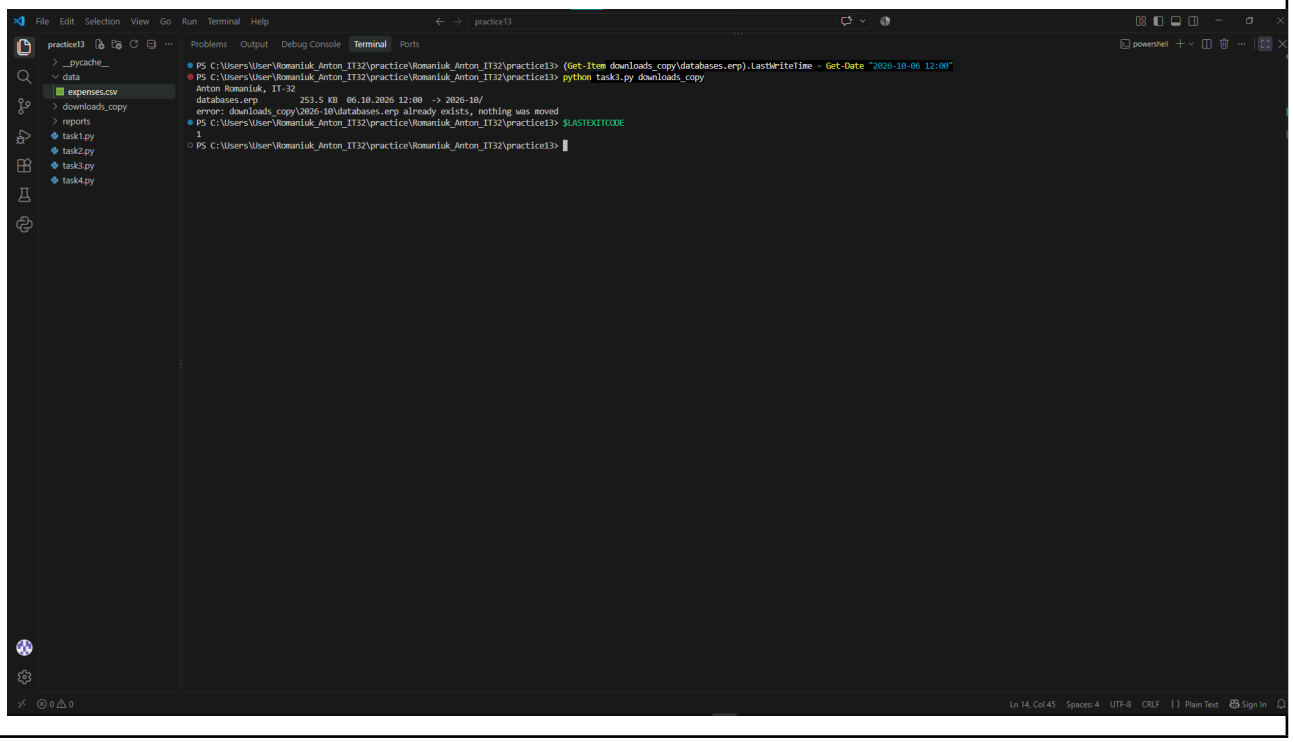


ls reports
cat reports\organize_*.txt



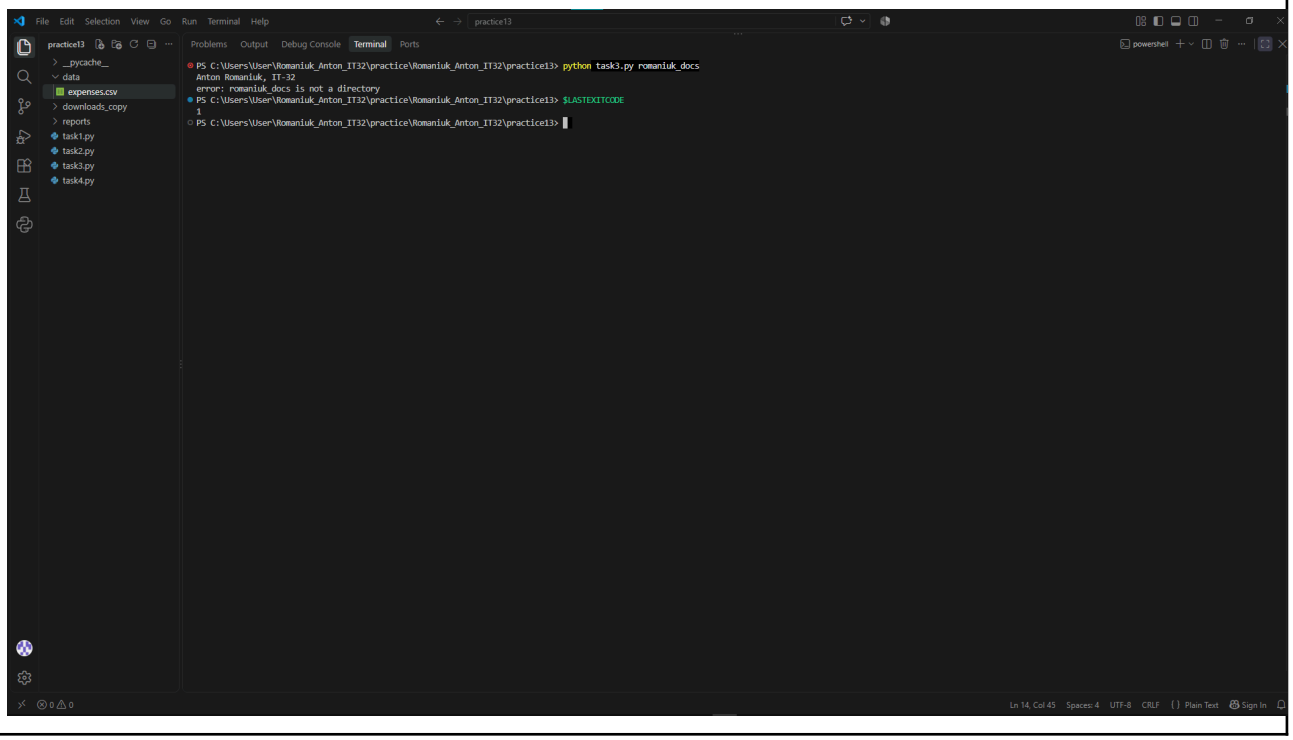
Прогін 3. Конфлікт імен (помилка, код 1, жоден файл не переміщено):

(Get-Item downloads_copy\<файл>).LastWriteTime = Get-Date "2026-10-06 12:00"
python task3.py downloads_copy
$LASTEXITCODE

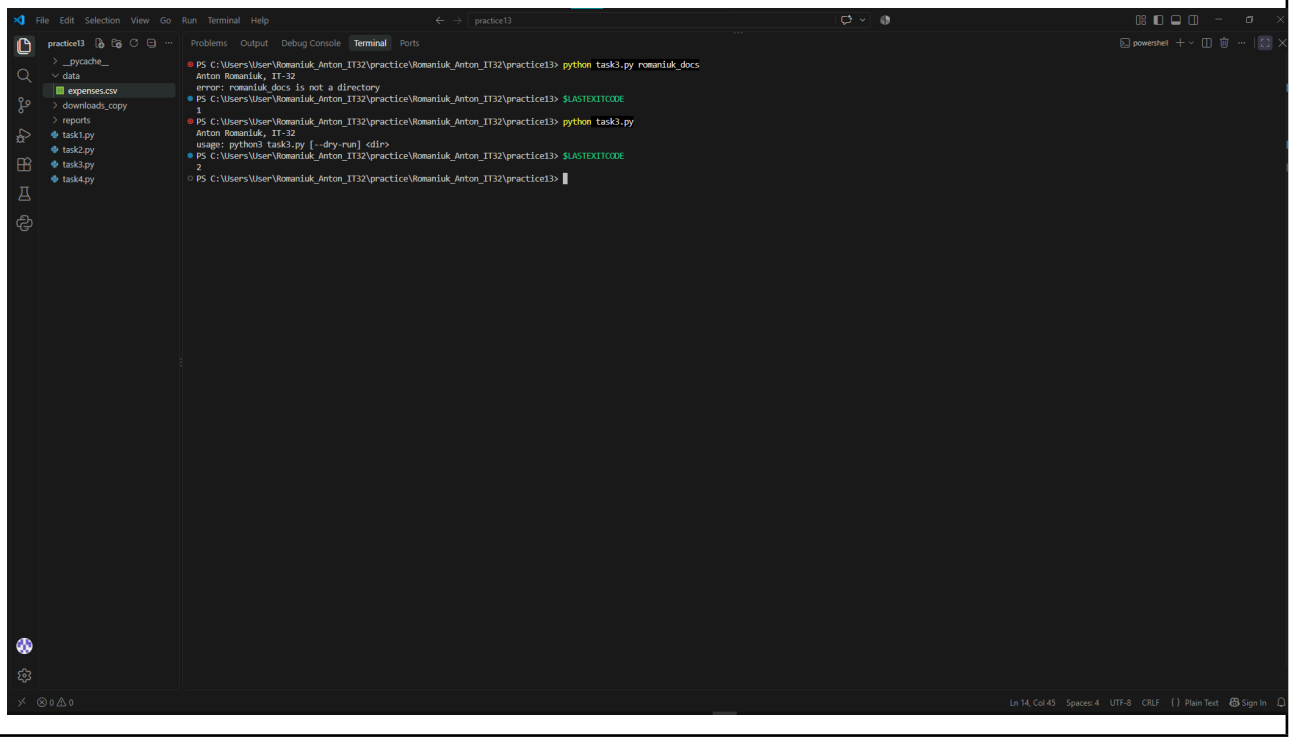


Прогін 4. Інші ситуації (таблиця кодів із завдання):

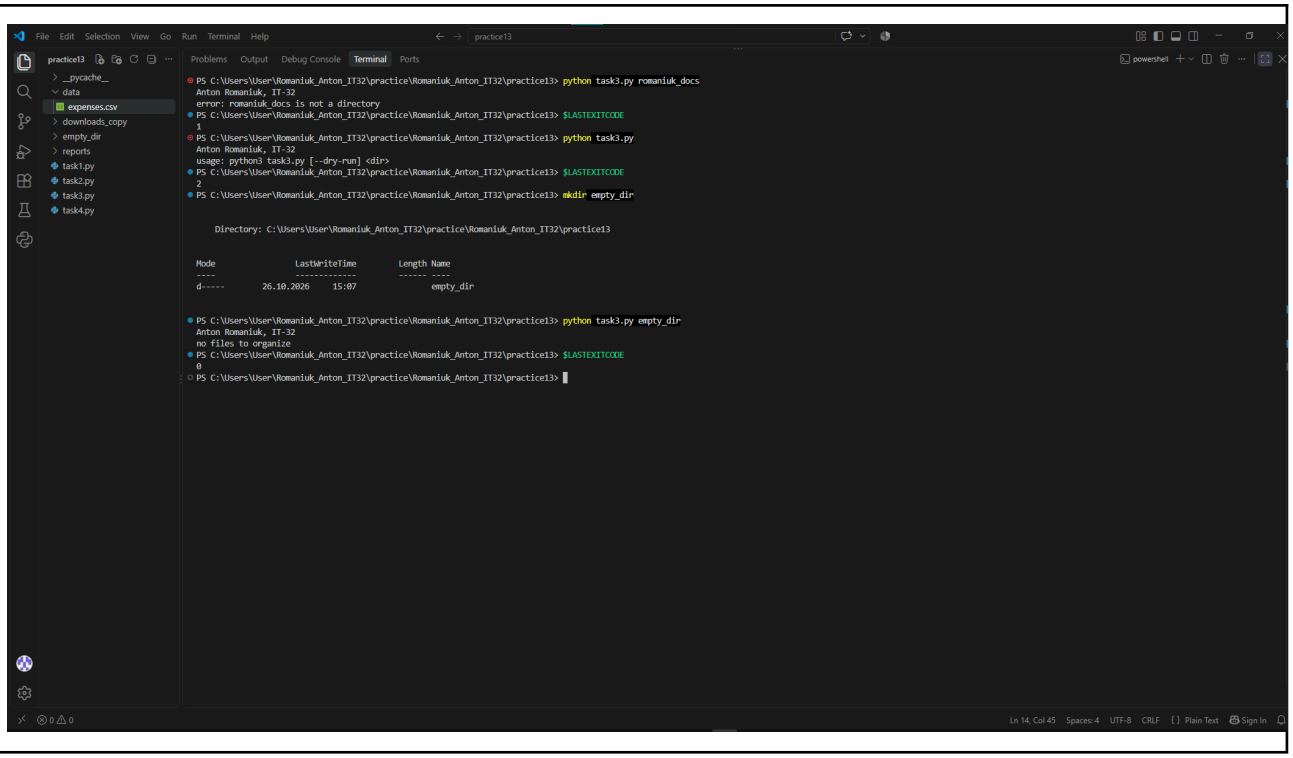
python task3.py romaniuk_docs
$LASTEXITCODE



python task3.py
$LASTEXITCODE



mkdir empty_dir
python task3.py empty_dir
$LASTEXITCODE

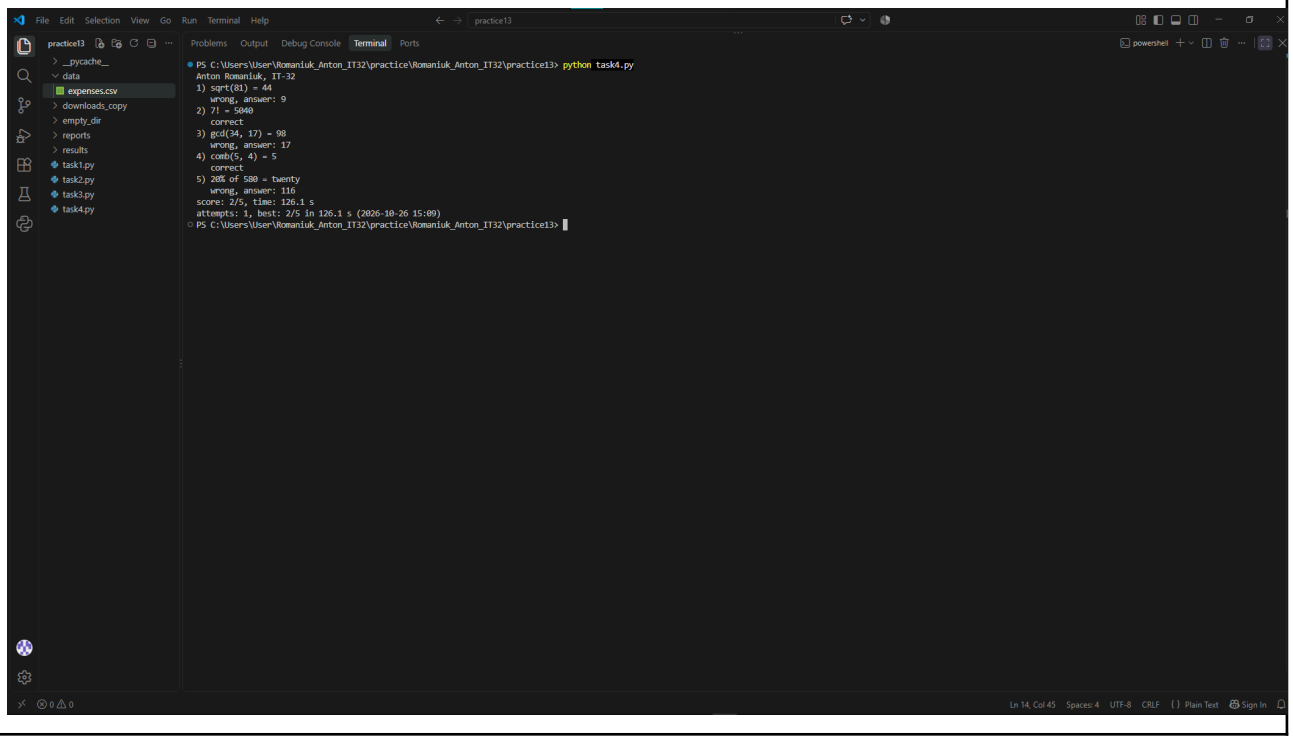


Завдання 4. Тренажер усного рахунку

Програма task4.py перед генерацією прикладів викликає random.seed(2008). Дає 5 прикладів (math.isqrt, math.factorial, math.gcd, math.comb, відсоток від числа), приймає відповіді з клавіатури, вимірює час через різницю двох datetime.now(), дописує результат у results/history.csv і виводить кількість спроб та найкращий результат (більше балів, за рівних балів — менше часу).

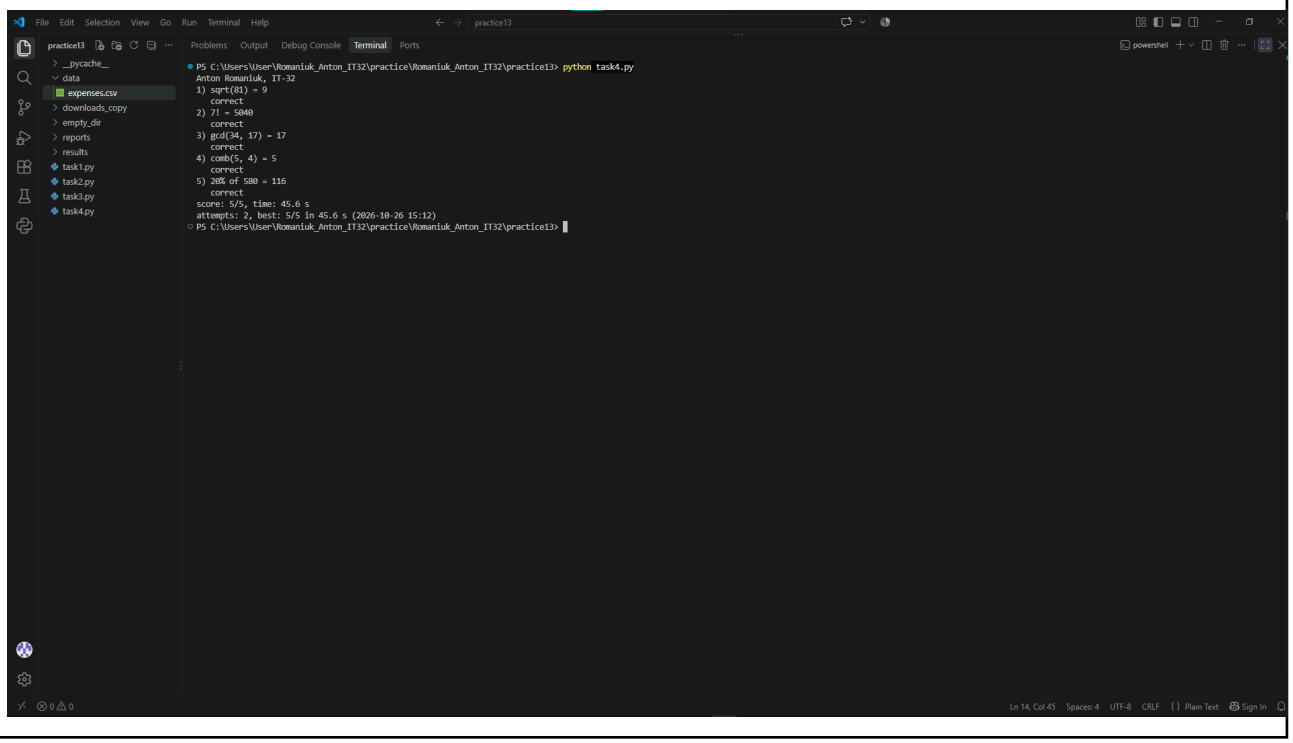
Запуск 1 (дайте кілька правильних і кілька неправильних відповідей, в одному прикладі введіть twenty):

python task4.py

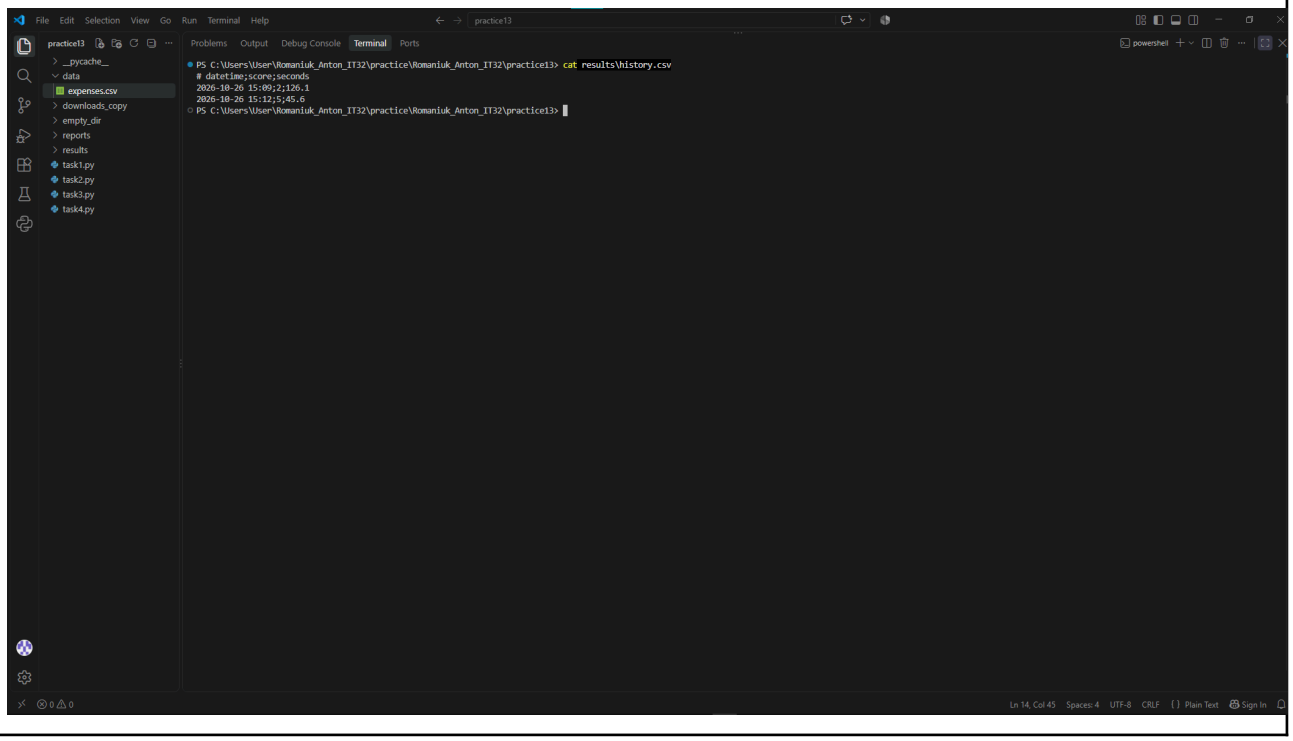


Запуск 2 (приклади мають бути **такі самі**, як у запуску 1; цього разу дайте правильні відповіді):

python task4.py



cat results\history.csv



Відповіді на контрольні запитання

1. Завдання 1: чому для кількості плиток і банок фарби не підходить round? Наведіть розміри, за яких round і math.ceil дають різний результат. Для покупки потрібне округлення **вгору**: якщо розрахункова кількість дробова, не вистачить навіть частки плитки чи банки, і доведеться докуповувати. round округлює до найближчого цілого, тому при дробовій частині менше 0.5 він округлює **вниз** і матеріалу не вистачає. (До того ж round у Python використовує банківське округлення: round(2.5) дає 2, а round(3.5) дає 4.)

Приклади з моїх даних: плитки 7.50 / 0.09 = 83.33: round і floor дають 83 (бракує 0.33 плитки), ceil дає 84. Фарба: 8.68 л / 2.5 л = 3.47 банки: round дає 3 банки (7.5 л — менше потрібних 8.68 л), ceil дає 4. Результати різні для будь-яких розмірів, при яких дробова частина кількості менша за 0.5 (і не нульова); при частці ≥ 0.5 обидві функції дають однаково.

math.floor теж не підходить: він завжди округлює вниз і гарантує нестачу.

2. Завдання 4: що зміниться, якщо прибрати random.seed? Чому з seed викладач може перевірити вашу програму, знаючи лише ваш рік народження? Без random.seed генератор ініціалізується випадково (за системним часом чи джерелом випадковості ОС), тому при кожному запуску числа в прикладах будуть різні. З random.seed(year) послідовність «випадкових» чисел детермінована: той самий seed завжди дає ту саму послідовність. Тому приклади в моїй програмі відтворювані, і викладач, знаючи лише мій рік народження (2008), може сам запустити алгоритм або свою версію і отримати ті самі приклади та правильні відповіді, а отже, перевірити, що програма справді працює з моїми персональними даними.

3. Навіщо в завданні 3 потрібен --dry-run і чому конфлікти перевіряються до переміщення першого файлу, а не під час?

--dry-run показує план переміщення без жодних змін на диску: можна перевірити, куди потраплять файли, і не ризикувати даними, особливо в реальному каталозі. Конфлікти (файл з таким ім'ям уже є в підкаталозі призначення) перевіряються **наперед, для всіх файлів**, щоб операція була «все або нічого». Якби конфлікт виявлявся під час переміщення, частина файлів уже опинилася б у нових місцях, а решта лишилася б на старих: каталог перебував би в напівпереробленому стані, який важко відкотити й легко зіпсувати. Попередня перевірка гарантує, що за помилки жоден файл не переміщено.